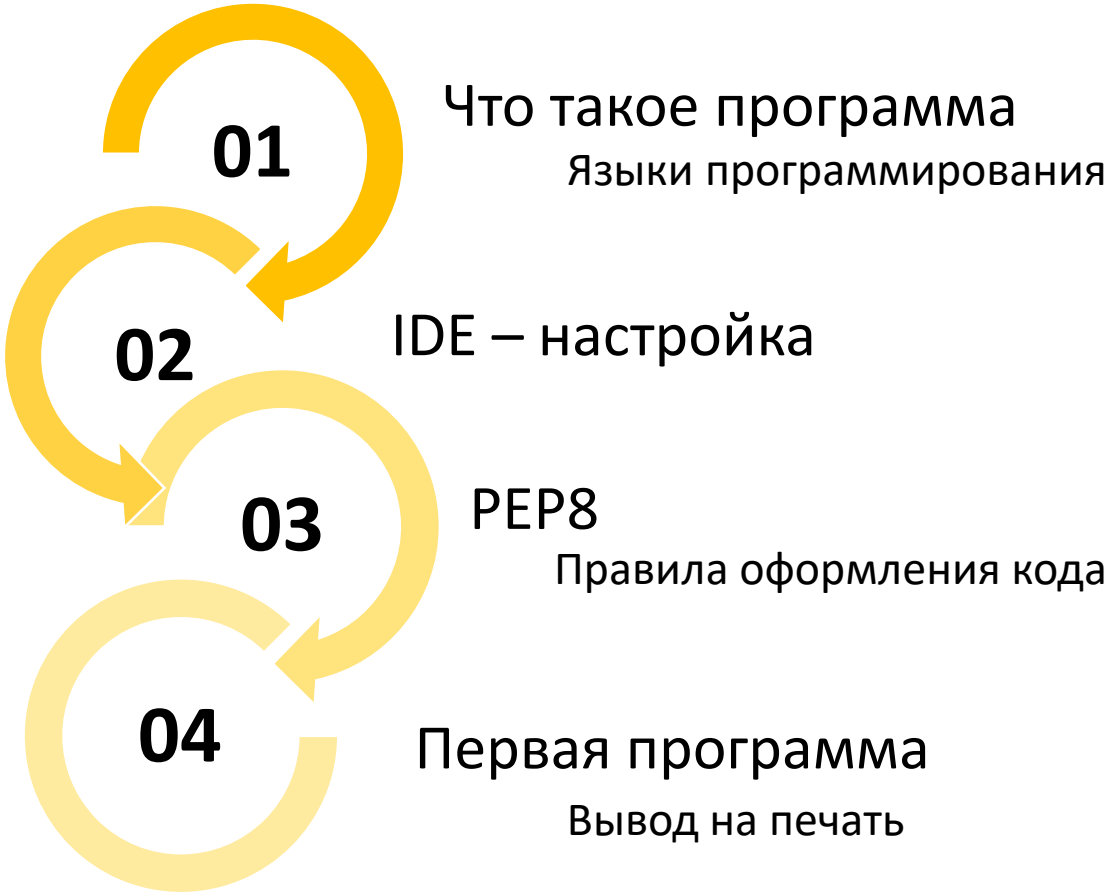

Яндекс

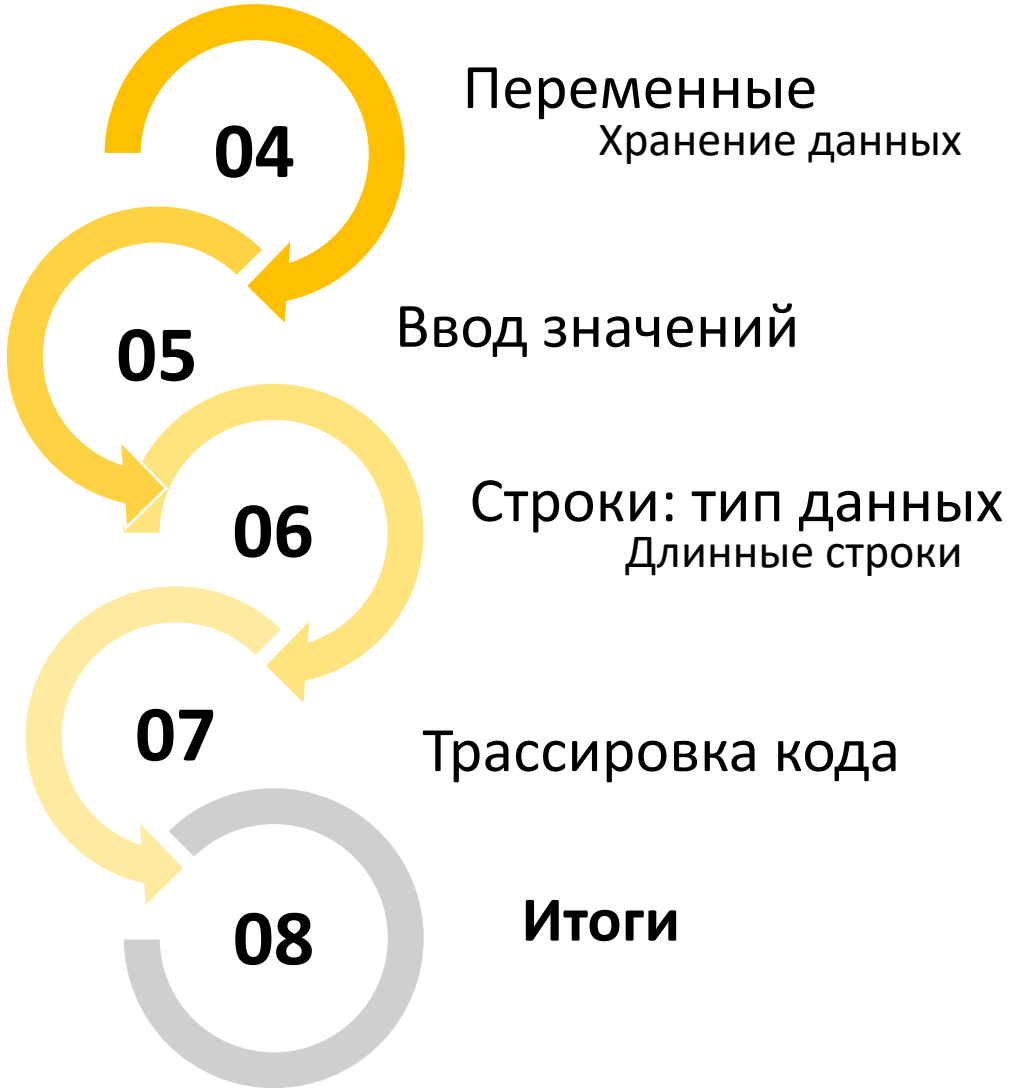
Яндекс Лицей

Первый контакт

Python как стиль жизни

Что впереди


01
Что такое программа
Языки программирования
02
IDE – настройка
03
PEP8
Правила оформления кода
04
Первая программа
Вывод на печать


04
Переменные
Хранение данных
05
Ввод значений
06
Строки: тип данных
Длинные строки
07
Трассировка кода
08
Итоги



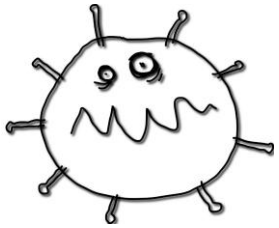
Что такое программа, или
Как заставить железку думать

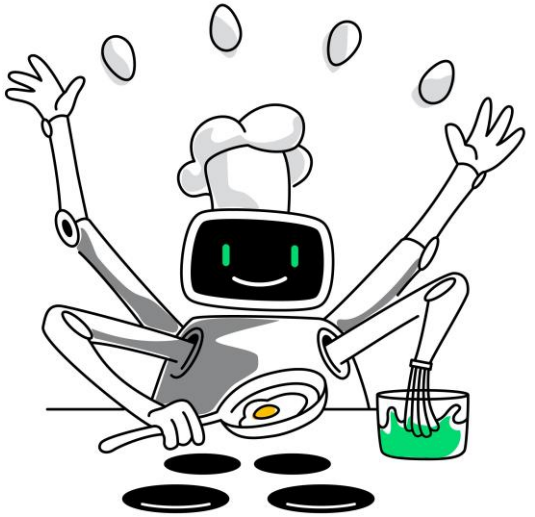
Много разных программ

- Браузер с котиками в интернете
- Медиаплеер
- Офисный пакет
- Вирус









Программа — это как рецепт для робота-кулинара.

Программа — это набор инструкций, понятных компьютеру

Программирование — это искусство писать инструкции так, чтобы любой компьютер понял, чего вы от него хотите.

Языки программирования: магия для своих

Немного истории

- 1940-е: Машинный код.
- 1950-е: Ассемблер.
- 1970-е: C.
- 1991: Гвидо ван Россум создал Python, назвав его в честь «Летающего цирка Монти Пайтона».

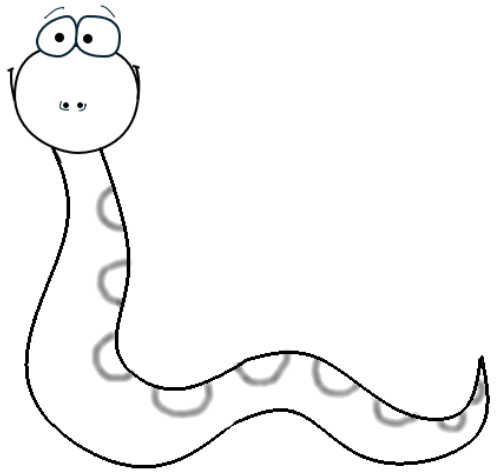


Компиляция vs интерпретация:

- **Компилируемые языки** (C++, Java): Вы пишете код → *компилятор* переводит его в машинный язык (или промежуточный код в случае с Java) раз и навсегда → программа запускается.
- **Интерпретируемые языки** (Python, JavaScript): *Интерпретатор* переводит код построчно во время выполнения.

Почему всё-таки Python?

- Лёгкий синтаксис, можно читать почти как текст на английском языке.
- Универсальность: веб, данные, AI, скрипты для автоматизации рутины — везде засветился.
- Огромное сообщество. Если вы застряли, какой-нибудь «Петя из Твиттера» уже решил вашу проблему.





IDE — ваша цифровая мастерская

Где писать программу

Исходный код программы — обычный текстовый файл. Программы на Python принято писать в файлах с расширением *.py*

Для разработки пользуются специальными программами, которые называются средами разработки (англ. **IDE**, *Integrated Development Environment*).

- **Wing IDE** — неплохая учебная среда, но не слишком хорошо подходит для настоящих проектов
- **PyCharm** — для тех, кто любит, чтобы IDE думала за них.
- **Visual Studio Code** — выглядит просто, но настраивается невероятно гибко.







Именно в VSCode мы и будем работать в этом курсе.

Первоначальная настройка,
или Готовимся к полёту



Устанавливаем Python с python.org

- **Важно:** галочка *Add Python to PATH* должна быть включена!
- Linux/MacOS-юзерам: Расслабьтесь. Python у вас уже есть.
- На Windows 11 можно открыть приложение командной строки и написать там *python* — тогда вас перенаправит в магазин приложений на установку Python

Примеры в курсе написаны с учётом версии интерпретатора 3.13

Качаем VSCode с официального сайта.

Рекомендуем создать на компьютере каталог, в котором вы будете хранить все свои программы.

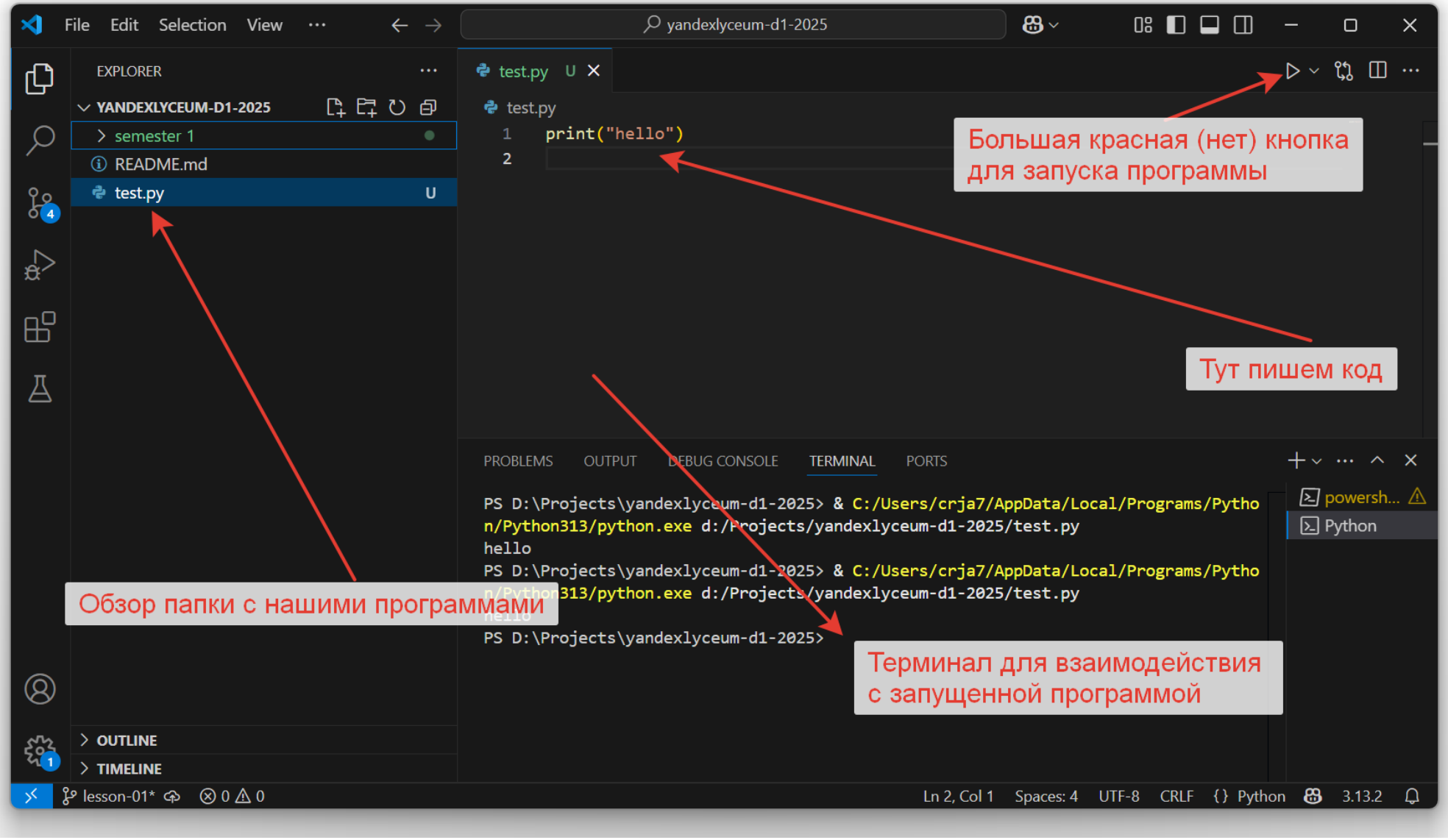


Открываем в VSCode каталог

в котором вы планируете работать. Когда вы создадите свой первый .py-файл, среда самостоятельно предложит вам установить все необходимые надстройки для комфортной работы.

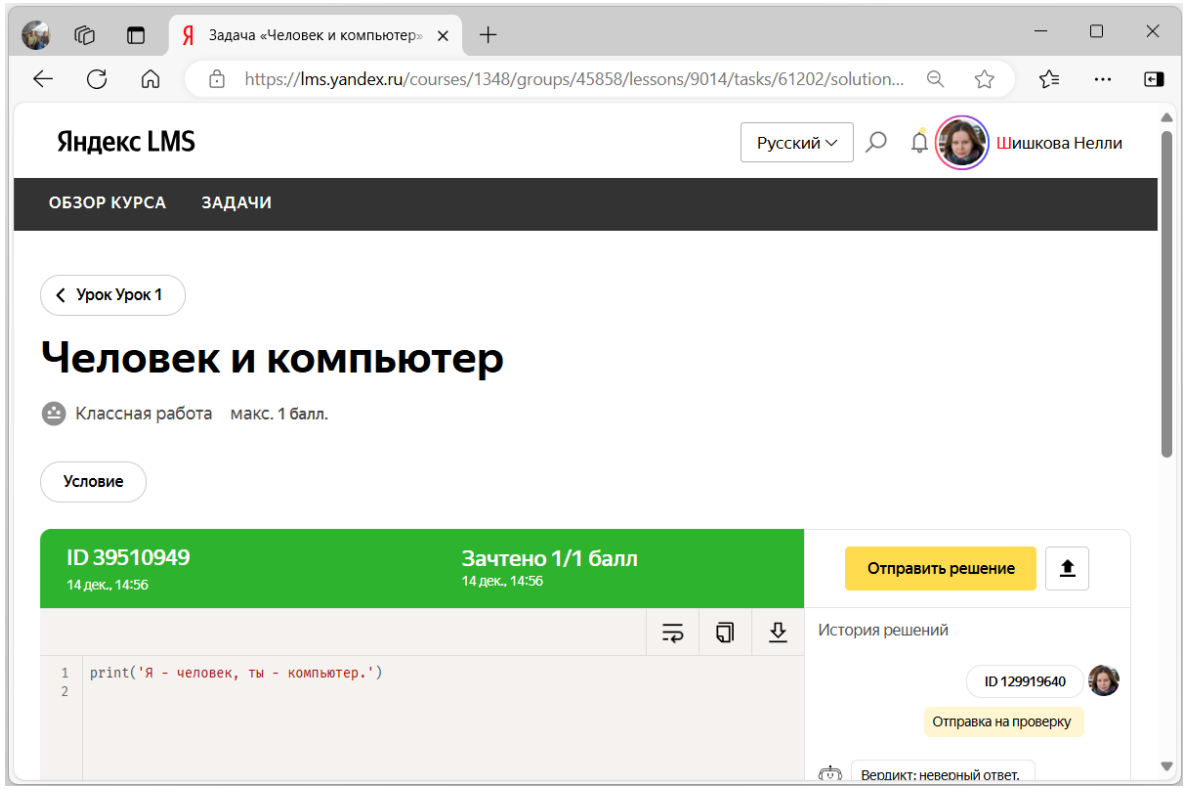
Соглашайтесь :)

Окно IDE VSCode



Тестирующая система

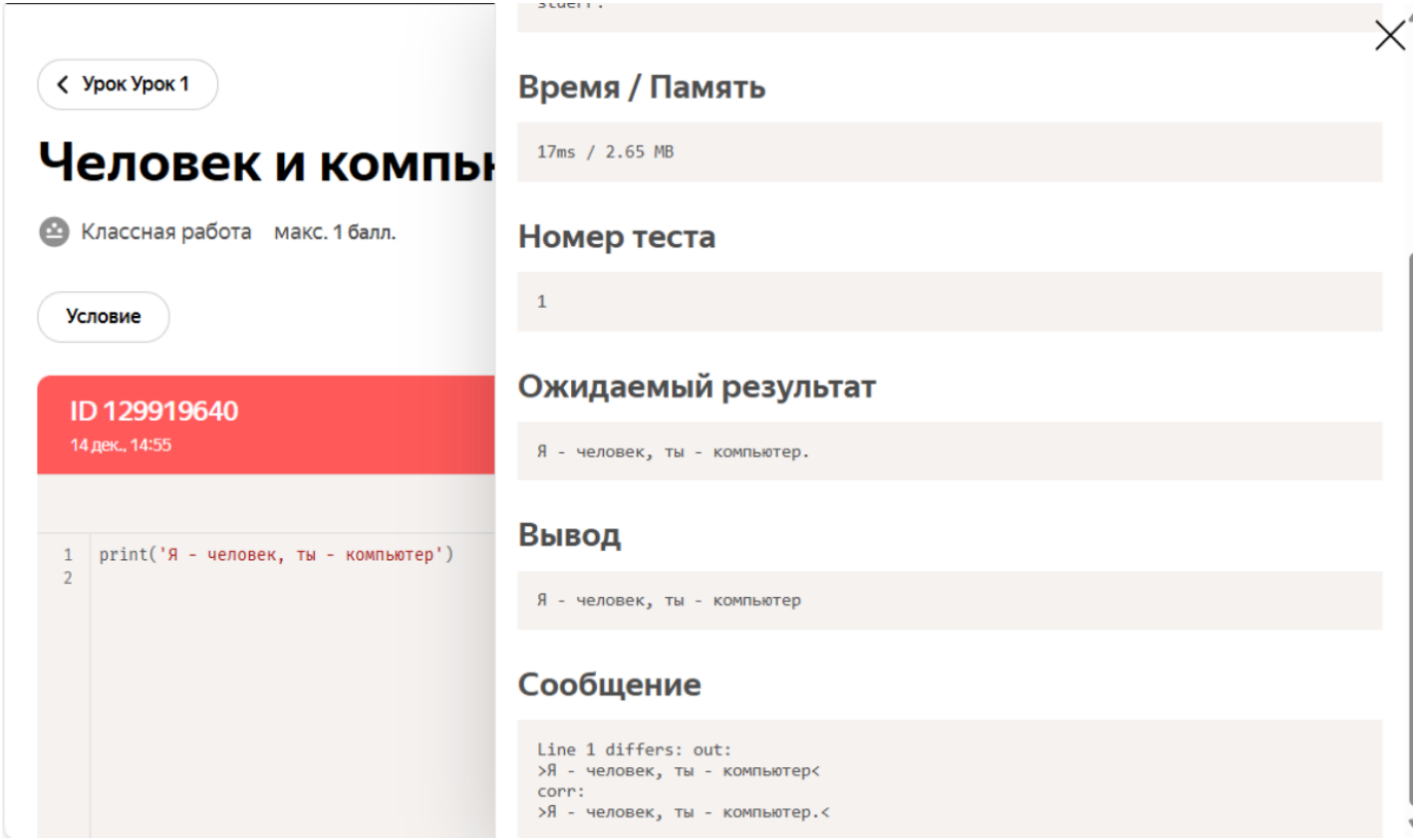
Задачи проверяются тестирующей системой. Для того чтобы сдать задачу на проверку, нужно открыть редактор, написать, вставить или загрузить из файла код и отправить его на проверку.



Если всё сделано правильно, полоса над редактором кода окрасится в приятный зелёный цвет, выставятся баллы за задачу, а в окошке чата задачи появится вердикт «ОК» и выставленные баллы.

Тестирующая система

Но если что-то пойдёт не так, то как узнать, что именно? Для этого можем обратиться к вердикту.



Самые частые вердикты с ошибками, которые вы будете видеть:

- Вердикт «неверный ответ» — значит, ваш код работает, но делает не то. Например, вместо «Hello, World!» вывел «Go away!».
- Вердикт «ошибка компиляции или оформления кода» — вы написали «pritn» вместо «print» или ваш код не соответствует стандарту PEP8.

Нажав на табличку со словом «вердикт» в чате задачи, можно посмотреть, какой вывод ожидала система и что вывела ваша программа (похоже, забыли поставить точку).

Оформление кода,
или Да кто он такой, этот ваш PEP8?



Любой язык — хоть русский, хоть язык программирования — это как игра в Lego. Есть кирпичики (слова/команды) и правила сборки (синтаксис).

При этом есть правила строгие и нестрогие:

- *Строгие правила*
- *Нестрогие правила*

PEP8 — это свод законов, как писать код, чтобы:

- **Вы** через месяц поняли, что написали.
- **Коллеги** не подумывали о ритуальном сожжении вашего кода.

Синтаксис Python — это как *грамматика*. Нарушил — программа не запустится.

PEP8 — как *правила этикета*. Нарушил — код будут читать, как расшифровывают почерк врача.



Вывод на печать

Print: «Мама, я в консоли!»

Python — регистрозависимый язык, то есть для него имеет смысл, большая буква или маленькая. Необходимо писать print только маленькими буквами.



Как это работает:

- Пишете print(— это как взять мегафон в руки.
- Внутрь скобок кладёте то, что нужно выкрикнуть: текст, числа, смайлики.
- Закрываете открытую скобку.
- Запускаете программу.
- Смотрите на результат в терминале.

```
print('Привет!')
```

Если хотим вывести какой-то текст, указываем его внутри кавычек. **Кавычки** могут быть как *одинарными*, так и *двойными*. Главное, чтобы текст начинался и заканчивался кавычками одного типа. И лучше использовать один тип кавычек по всей программе.

Ломаем программу

PEP 8

Избегайте использования пробелов сразу перед открывающей скобкой, после которой начинается список аргументов функции.

Правильно:

```
print('Привет!')
```

Неправильно:

```
print ('Привет!')
```

Давайте попробуем намеренно сломать нашу программу и посмотрим, как на это отреагирует Python. Для начала допустим опечатку в имени команды print.

```
pront('Привет!')
```

Мы увидим, что VSCode не узнал команду и не выделил её цветом. Если же мы запустим программу с этой ошибкой, то получим:

```
NameError: name 'pront' is not defined. Did you mean: 'print'?
```

Конечно, если ошибка больше, чем в один символ, то интерпретатор будет бессилен и просто скажет, что не знает такой команды.
Теперь давайте забудем закрыть кавычку у строки:

```
print('Привет!)
```

```
SyntaxError: unterminated string literal
```

Интерпретатор нам прямо говорит, что мы забыли закрыть кавычку и он не понимает, где заканчивается строка.



Печатаем, как профессиональная типография

Печатаем строки

Давайте сделаем так, чтобы следующая программа выводила две строчки с текстом.
Для этого нам понадобится ещё одна команда print. Эту команду обязательно надо написать с новой строки. **Каждая новая команда пишется с новой строки!**

```
print('Привет!')

print('Наша вторая программа в два раза полезнее первой!')
```

Сохраните программу, запустите и проверьте.

Печатаем строки

Проведём смелый эксперимент: разобьём строчку во втором print на несколько, каждую часть напишем в своих кавычках, а сами части перечислим через запятую, вот так:

```
print('Наша', 'вторая', 'программа', 'в', 'два', 'раза', 'полезнее', 'первой!')
```

```
Наша вторая программа в два раза полезнее первой!
```

Всё, что мы перечислим внутри круглых скобок команды print через запятую, будет выведено через пробел.
Если быть совсем честными, то не через пробел, а через разделитель, который по умолчанию — пробел.

PEP 8

После запятой обязательно нужно добавлять пробел (как и при написании текста на русском языке)

Правильно:

```
print('Привет', 'Яндекс!')
```

Неправильно:

```
print('Привет','Яндекс!')
```

Печатаем (не только) строки

Несколько примеров:

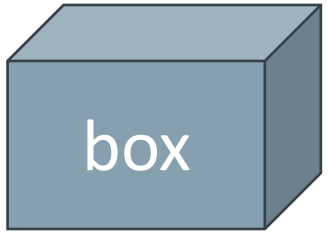
```
print('123')  # -> 123
print ('Hello, world!')  # -> Hello, world!
print('Hello,', 'world!')  # -> Hello, world!
print(123)  # -> 123
print(31, 'декабря наступает Новый Год!')  # -> 31 декабря наступает Новый Год!
print()  # -> Просто перенос строки
print(2, '+', 3, '=', 2 + 3)  # -> 2 + 3 = 5
```



Переменные –
память нашей программы

Переменные

Можно представить, что **переменная — это коробка с наклейкой**. Можете положить в неё что угодно: число, текст, список котиков.



У любой переменной есть имя и значение:

- Имя переменной не должно начинаться с цифры и может состоять из букв, цифр и символа подчёркивания. **Не используйте никакой язык, кроме английского, пожалуйста!**
- Значение переменной — то, что сохраняет в себе переменная.

Переменные

PEP 8

Для именования переменных принято использовать стиль *lower_case_with_underscores* (слова из маленьких букв с подчеркиваниями). Избегайте использовать такие символы, которые могут неоднозначно трактоваться в различных шрифтах: это буква **O** (большая и маленькая) и цифра **0**, буква **I** (большая и маленькая), буква **L** (большая и маленькая) и цифра **1**.
Нельзя использовать в качестве имени переменной и ключевые слова, которые существуют в языке.

```
# хорошие имена переменных
pi = 3.14
cats_count = 10
username = 'Иван'

# плохие имена переменных
cats_count = 'Иван'
z = 3.14
dfogjsojdfgsoijgf = 10
```

Правильно:

```
bird = "Тук-тук"
```

Неправильно:

```
bird="Тук-тук"
```

Всегда окружайте оператор присваивания одним пробелом с каждой стороны:

Знак «=» обозначает команду под названием «присваивание». Оператор присваивания присваивает значение, которое находится справа от знака равно, переменной, которая находится слева от знака равно.

Переменные

Так как Python — язык с динамической типизацией, нам не надо задумываться, что за значение мы хотим положить в переменную и какое там лежало раньше.

Мы присваиваем значение переменной и потом всегда сможем получить это значение по её имени.

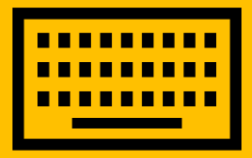
```
parrot_name = 'Кеша'
print('Мы купили попугая и назвали его', parrot_name)
print(parrot_name, 'умеет говорить несколько фраз')
```

Мы можем в любой момент времени изменить значение переменной на другое.

```
cat_name = 'Пудинг Иванович'
print('Моего кота зовут', cat_name)
cat_name = 'Поганец'  # После выполнения этой строчки по имени cat_name будет храниться уже другое значение
print('Но для друзей он —', cat_name)
```

Если у нас имеется переменная, мы можем делать с её значением всё что угодно, например присвоить другой переменной:

```
hello = 'Здравствуйте.'
hello2 = hello
print(hello2)
```



Команда input() —

налаживаем диалог с пользователем

Ввод значений – input()

Пока что наши программы напоминали заученные монологи: они выводили в мир заранее прописанные фразы, как робот-лектор. Но настоящая магия начинается, когда программа научится слушать.

```
print('Как тебя зовут?')   # Программа спрашивает
name = input()             # Программа замолкает и ждёт...
print('Привет,', name)     # Программа реагирует на ответ
```

Запустите этот код. Если в консоли появилась тишина после вопроса — это не баг. Это программа вежливо ждёт, пока вы не введёте имя и не нажмёте Enter.

Как это работает:

- input() — это команда-ловушка. Когда программа до неё доходит, она замирает и смотрит в окно терминала, как кот, ждущий мышь.
- Пользователь печатает что-то (например, «Бэтмен») и жмёт Enter.
- Всё, что было введено, подменяет собой input(). Теперь name = 'Бэтмен'

Ввод значений – input()

Ещё примеры:

```
print('Какая твоя любимая еда?')
meal = input()
print('Да.', meal, '— это вкусно.')
```

```
print('Как тебя зовут?')
name = input()  # Ждём, пока пользователь введёт имя
print('Привет,', name)
print('А какая твоя любимая еда?')
meal = input()  # Ждём, пока пользователь введёт любимое блюдо
print('Да.', meal, '— это вкусно.')
```

Обратите внимание: интерпретатор ждёт, что пользователь что-то введёт с клавиатуры ровно столько раз, сколько команд input() встречается в программе. Каждый input() завершается нажатием Enter на клавиатуре.

Вне зависимости от того, что ввёл пользователь, input() **всегда возвращает строку**, то есть даже если на вход пришло число 42, то в переменную будет записана строка «42». Что с этим делать, мы уже очень скоро узнаем.



Немного о строках

Кавычки — сколько их? Одна? Две? Три?

Строковые значения должны быть заключены в кавычки, иначе интерпретатору будет невозможно их отличить от имён переменных или от команд самого языка. Это могут быть:

- **Одинарные:** 'Привет'
- **Двойные:** "Привет"

Разницы нет, но стиль лучше соблюдать единый.

Но можно ещё использовать три одинарные или три двойные кавычки для многострочных строковых данных:

```
poem = """Розы красные,
Фиалки синие,
Я пишу код,
И мне хорошо."""
```

Длинные строки — как не потеряться в тексте

PEP8

Длина строки с кодом не должна быть более 79 символов. Это правило придумали во времена маленьких мониторов, поэтому сейчас почти всегда делается послабление. В нашей проверяющей системе максимальная длина строки — 110 символов.

Но что, если мы хотим вывести текст, который точно длиннее 110 символов? Не надо паниковать: для решения этой проблемы есть несколько способов:

- Допустимый, но нежелательный **перенос через \:**

```
long_text = "Это очень длинная строка, которую \
удобно разбить на две линии."
```

- Прекрасный и удобный с помощью **скобок:**

```
long_text = (
    "Этот способ даже лучше — "
    "не нужен символ переноса."
)
```

Комментарии — заметки для себя и потомков



Комментарии бывают нескольких видов:

- Однострочные комментарии:

```
# Это важный комментарий. Не удалять!
x = 5  # Волшебная константа
```

- Многострочные:

```
'''
Многострочные комментарии пишутся
именно в трёх одинарных кавычках
Не будут ошибкой и три двойные
Но так не принято в приличном обществе
'''
```

Зачем?

- Объяснить сложную логику.
- Временно отключить код (закомментировать).
- Напомнить себе, что тут происходит (через месяц всё забудется).

Не пишите очевидные комментарии вроде

```
x = 5  # Присвоить x значение 5
```

Лучше объясните, зачем это нужно:

```
# Рассчитать дискриминант квадратного уравнения
D = b ** 2 - 4 * a * c
```

Ещё пример по шагам



Разберём код:

```
a = 5
b = a  # b = 5. Нет, это не создаёт связь навеки!
a = 10
print(a + b)  # Выведет 15. Да, переменные живут своей жизнью.
```

Таблица трассировки для чайников:

Шаг	Код	Переменная `a`	Переменная `b`	Состояние программы
1	`a = 5`	5	не определена	«А что такое b?»
2	`b = a`	5	5	«Боже, они одинаковые!»
3	`a = 10`	10	5	«b, ты предатель!»
4	`print(a + b)`	10	5	«Так, надо разобраться, что нужно вывести»
4.1	`a + b`	10	5	15. Можно удалять приложение «Калькулятор»
4.2	`print(15)`	10	5	Вывод: 15. Все в шоке.

Итог: Вы уже программисты!



Поздравляем! Вы прошли первый уровень Python-вселенной:

01 Настроили VSCode и теперь можете кодить на стиле.

02 Узнали, как общаться с Python через print() и input().



03 Усвоили, что код должен быть красивым, как котик в смокинге (спасибо PEP8).

P. S. Если устали — поспите, работает так же эффективно, как перезагрузка компьютера.